



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Beezie Claw

July 2026

Prepared by
Mr Auditor
ZanyBonzy

Introduction

A time-boxed security review of the **Beezie** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A code review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About Beezie

Beezie's solana on-chain logic, comprises two complementary programs, beezie-claw and beezie-redemption. beezie-claw implements the core "claw machine" pack-opening game via a commit-reveal pattern, i.e users pay to open packs commit (reserving packs and storing a seed commitment), then an authorized backend cranker runs the reveal to select and deliver Metaplex Core NFTs, with off-chain randomization. It also covers machine lifecycle and includes various admin controls and buybacks. beezie-redemption lets users trade Core NFTs plus payment tokens into vault wallets. Documentation under docs/ is thorough, spanning architecture, invariants, glossary, per-instruction specs, user flows, and state-account references.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

review commit hash - [ef33235ce4c64f2a11431259a132d4543987ad81](#)

Scope

The following folders and every file in them were in scope of the audit:

- `./colony-mono/solana-programs/programs/beezie-claw`
- `./colony-mono/solana-programs/programs/beezie-redemption`

The following number of issues were found, categorized by their severity:

- Critical & High: 0 issues
- Medium: 1 issues
- Low and Informational: 7 issues

Findings Summary

ID	Title	Severity	Status
[M-01]	A single user can reuse a one-per-user discount code many times by firing multiple concurrent play/build requests, draining its shared budget	Medium	Fixed
[L-01]	<code>price = 0</code> is accepted by <code>create_machine</code> and <code>update_price</code> , yielding free plays	Low	Acknowledged
[L-02]	Buyback net-payout allows a 0-amount payout at 100% fees	Low	Fixed
[L-03]	Asymmetric Pause Gate: Fee Config Updates Allowed Without Pausing, Unlike Price Updates	Low	Acknowledged
[L-04]	Redundant Bump Rewrite on Every Call in <code>init_if_needed</code> Handlers	Low	Acknowledged
[L-05]	No Instruction to Close <code>PlayerState</code> : User Rent Permanently Locked	Low	Acknowledged
[I-01]	Redundant <code>Pubkey::default()</code> Check on <code>new_payment_mint</code> in <code>update_config</code>	Informational	Fixed
[I-02]	<code>initialize_factory</code> Upgrade-Authority Check Degenerates to <code>admin == default</code> if the Program Is Made Immutable Before Init	Informational	Acknowledged

Detailed Findings

[M-01] A single user can reuse a one-per-user discount code many times by firing multiple concurrent play/build requests, draining its shared budget

Severity

Impact:

Medium

Likelihood:

Medium

Description

On-chain, `DiscountCode.remaining_uses` is a global cross-user counter — the struct has no user field and the PDA is machine-agnostic (`[b"discount", code_hash]`), so the program has no notion of "this user used this code":

```
// programs/beezie-claw/src/state/discount.rs
pub struct DiscountCode {
    pub code_hash: [u8; 32],
    pub discount_bps: u16,
    /// Remaining global uses. Decrementd once per successful discounted
    commit.
    pub remaining_uses: u32,
    pub active: bool,
    pub bump: u8,
}
```

`resolve_discount` (`open_packs_commit.rs`) decrements it once per commit and applies one pack's discount regardless of `quantity`. The per-user cap lives entirely in the backend as check-then-act, not an atomic reservation: `POST /claw/play/build` requires the caller to have claimed the code via `/claw/apply-promo-code` (which creates a per-user `Discount` row, `totalUses: 1`) and validates `remainingUses > 0` — a pure read — before `play_signer` co-signs the commit. The row is decremented only later, asynchronously, when the `PacksCommitted` event is ingested (`recordSolanaDiscountUse` in `db/mutations/discount/updateDiscount.ts`, keyed by the on-chain nonce for idempotency).

Because nothing reserves or locks the row at co-sign time, a user firing N concurrent `/play/build` requests before the first event settles passes the check N times and receives N co-signed discounted commits, despite a nominal 1-use-per-user policy. The only synchronous backstop is the on-chain global `remaining_uses`, so the excess is drawn from other users' share of the budget. The on-chain sequencing guard does not help: `PendingCommitExists` is keyed on `PlayerState [b"player", machine_id, user]`, serializing only same-`(machine, user)` commits; the race parallelizes across machines, and the code is machine-agnostic. The atomic `reserveDiscountForSigning` primitive exists but is dead on Solana — reachable only via the EVM-style `/claw/discount-signature` path, gated by `checkDiscountFundingAvailability` (a stub returning `false`) and `getDiscountSignature` (throws unconditionally). A per-user/IP rate limiter (`playRateLimiter`, 20 pulls/60s) caps a single burst to ~20 co-signs, but a patient wallet drains the full budget across successive windows.

Note: This per-user cap is not a specified invariant in docs. The "one use per user" intent exists solely in `lucky-api-solana`, as the `totalUses: 1` field written on the per-user `Discount` row (`/claw/apply-promo-code; services/email/lifecycle/createLifecycleDiscount.ts`) and the `remainingUses > 0` read that gates it. Because the on-chain layer and its spec only ever model a global budget, the per-user policy has no authoritative definition and is enforced in the one place — the backend co-signer — that cannot enforce it atomically.

This is bounded promo-budget leakage, not fund extraction: every burned use still rides a real, paid pack purchase, and total exposure never exceeds the configured global budget — `remaining_uses × bps/10_000 × max(machine.price)`, since `machine.price` is admin-mutable (`update_price`) and the code is machine-agnostic. The bug changes who captures the budget and lets the intended one-per-user cap be exceeded.

Recommendations

Fix is not trivial as enforcement exists offchain, but if per-user enforcement must be trustless, recommend move usage tracking on-chain (e.g. a `[b"discount_use", code_hash, user]` PDA created on first redemption) so the program itself rejects a second use by the same wallet, rather than relying solely on the backend co-signer.

[L-01] `price = 0` is accepted by `create_machine` and `update_price`, yielding free plays

Description

`handle_create_machine` (`create_machine.rs:105`) and `handle_update_price` (`admin/update_price.rs:28`) store `price / new_price` with no positivity guard, even though `create_machine` carefully null-guards the three identity fields (`machine_admin`, `play_signer`, `collection` — `create_machine.rs:68–79`) and validates the fee config. In `open_packs_commit`, `price = 0` makes `gross_amount = machine.price * quantity = 0`; the payment CPI is wrapped in `if amount > 0` (`open_packs_commit.rs:142`) and is therefore skipped, while pool slots are still reserved (`effective_pool_size -= quantity`) and real NFTs are later handed out at reveal. The user receives the NFT having paid nothing.

Recommendations

Add `require!(price > 0, BeezieClawError::ZeroPaymentAmount)` (or a dedicated `InvalidPrice`) in both `handle_create_machine` and `handle_update_price`, mirroring the existing `quantity > 0 / payment_amount > 0` guards.

[L-02] Buyback net-payout allows a 0-amount payout at 100% fees

Description

In `handle_buyback`, each fee transfer is guarded by `if fee_amount == 0 { continue; }` (`buyback.rs:178`), but the final net payout to the user (`buyback.rs:196`) is unconditional. `validate_fee_config` permits `total_bps <= 10_000` (`initialize_factory.rs:85`), i.e. fees summing to 100% pass validation. With a 100%-fee config (or when rounding drives the remainder to zero), `net_payment_amount = 0` and the user surrenders their NFT(s) to the manager wallet for a 0-value payout.

Recommendations

Reject transactions if `net_payment_amount == 0`

[L-03] Asymmetric Pause Gate: Fee Config Updates Allowed Without Pausing, Unlike Price Updates

Description

`update_price` requires the machine to be paused (`constraint = machine.paused`), but `update_machine_fee_config` (which mutates buyback fee wallets and fee bps on the same `Machine` account) has no pause or `is_finished` gate and can be executed at any time by the machine admin or factory admin. The asymmetry is not documented in `docs/architecture.md` or in the instruction source. An admin can change fee recipients or fee bps on a live machine while plays are in flight. Buybacks executed after the change use the new fee routing, not the configuration in effect when the play was committed.

```
// programs/beezie-claw/src/instructions/admin/update_price.rs, line 15
constraint = machine.paused @ BeezieClawError::MustBePausedToChangePrice,

// programs/beezie-claw/src/instructions/admin/update_machine_fee_config.rs, lines 29-40
pub fn handle_update_machine_fee_config(
    ctx: Context<UpdateMachineFeeConfigAccountConstraints>,
    fee_wallets: [Pubkey; 3],
    fee_bps: [u16; 3],
) -> Result<()> {
    validate_fee_config(&fee_wallets, &fee_bps)?;
    let machine = &mut ctx.accounts.machine;
    machine.fee_wallets = fee_wallets; // no paused / is_finished check
    machine.fee_bps = fee_bps;
    // ...
}
```

Recommendations

It is recommended to add a pause gate for consistency with `update_price` (`constraint = machine.paused`).

[L-04] Redundant Bump Rewrite on Every Call in `init_if_needed` Handlers

Description

Three handlers that use `init_if_needed` PDAs unconditionally rewrite the account's `bump` field on every invocation: `set_discount_code` (beezie-claw), `open_packs_commit` (beezie-claw), and `set_quote` (beezie-redemption). The bump is a property of the PDA derivation and never changes after the account is first initialized, so every write after the first stores the identical value. The redundancy is harmless today (`ctx.bumps` always yields the same canonical bump), but the pattern obscures which fields are genuinely mutable and adds a needless store on the hot commit path.

```
// programs/beezie-claw/src/instructions/admin/set_discount_code.rs, line 57
discount.bump = ctx.bumps.discount_code;

// programs/beezie-claw/src/instructions/open_packs_commit.rs, line 225
player.bump = ctx.bumps.player_state;

// programs/beezie-redemption/src/instructions/set_quote.rs, line 87
quote.bump = ctx.bumps.quote;
```

Recommendations

Write the bump only on first initialization: guard with a zero check (`if account.bump == 0 { account.bump = ctx.bumps.<account>; }`) or set it in a dedicated create path, so update invocations do not touch the field. Alternatively, document that the rewrite is an intentional idempotent no-op.

[L-05] No Instruction to Close `PlayerState`: User Rent Permanently Locked

Description

`PlayerState` is created via `init_if_needed`, `payer = user` in `open_packs_commit`, so each user funds the rent-exempt reserve for their own per-(machine, user) PDA. The program exposes no instruction to close this account and refund the rent, even after the machine is stopped (`is_finished == true`) and no further plays can occur. The account and its locked rent persist for the life of the deployment with no recovery path.

```
// programs/beezie-claw/src/instructions/open_packs_commit.rs, lines 48-59
#[account(
    init_if_needed,
    payer = user,
```

```

    space = 8 + PlayerState::INIT_SPACE,
    seeds = [
        PlayerState::SEED_PREFIX,
        machine.machine_id.to_le_bytes().as_ref(),
        user.key().as_ref(),
    ],
    bump,
)]
pub player_state: Box<Account<'info, PlayerState>>,
// No close_player_state instruction exists anywhere in instructions/

```

Each distinct user who plays a machine strands ~0.0017 SOL (rent-exempt reserve for a 115-byte account) with no way to reclaim it, not even after the machine reaches its terminal `is_finished` state via `stop`. Purely economic: no fund loss beyond the user's own locked rent, no DoS, no relaunch/seed-collision risk (`machine_id` is monotonic, so a stopped machine's `PlayerState` PDAs can never block or shadow a future machine). Impact scales linearly with unique-user count across the program's lifetime, never per-play.

Recommendations

It is recommended to add a permissionless `close_player_state` instruction, callable once the parent machine is finished, that closes the caller's own `PlayerState` PDA and refunds rent via Anchor's `close = user`.

[I-01] Redundant `Pubkey::default()` Check on `new_payment_mint` in `update_config`

Description

In `beezie-redemption`'s `update_config`, the `new_payment_mint` branch re-checks `new_payment_mint != Pubkey::default()` even though that condition is already guaranteed unreachable by the mint-account pairing match executed earlier in the same handler. To reach the branch with `new_payment_mint == Some`, control must have passed the `(Some(pk), Some(mint_acc))` match arm, which enforces `mint_acc.key() == pk` and `mint_acc.owner == anchor_spl::token::ID`, i.e. a real classic-SPL `Mint` account. `Pubkey::default()` (all-zeros) can never be a token-program-owned `Mint`, so the non-default check can never fire. Dead guard.

This is asymmetric with the other rotated fields: `new_admin`, `new_payment_vault`, `new_collection`, `new_redeem_signer`, and `new_redemption_nft_receiver` are raw `Pubkey` args with **no** backing account validation, so their `Pubkey::default()` checks are load-bearing and must stay. Only the `payment_mint` one is redundant.

```

// programs/beezie-redemption/src/instructions/update_config.rs, lines 69-
77 (the guaranteeing match arm)
(Some(pk), Some(mint_acc)) => {
    require!(mint_acc.key() == pk, RedemptionError::InvalidPaymentMint);
    require!(

```



```

        mint_acc.to_account_info().owner == &anchor_spl::token::ID,
        RedemptionError::UnsupportedPaymentMint
    );
}

// programs/beezie-redemption/src/instructions/update_config.rs, lines 99-
// 105 (the redundant check)
if let Some(new_payment_mint) = args.new_payment_mint {
    require!(
        new_payment_mint != Pubkey::default(), // unreachable-fail: a
        token-owned Mint is never the default pubkey
        RedemptionError::InvalidPaymentMint
    );
    config.payment_mint = new_payment_mint;
}

```

Recommendations

It is recommended to drop the `Pubkey::default()` check inside the `new_payment_mint` branch and assign directly

[I-02] `initialize_factory` Upgrade-Authority Check Degenerates to `admin == default` if the Program Is Made Immutable Before Init

Description

The `initialize_factory` admin constraint compares the signer to `program_data.upgrade_authority_address.unwrap_or(Pubkey::default())`. When the program's upgrade authority is `None` (program made immutable), the `unwrap_or(Pubkey::default())` fallback reduces the check to `admin.key() == Pubkey::default()`. No keypair can sign as the all-zero pubkey, so factory initialization becomes permanently impossible. The failure is silent: a degenerate comparison rather than an explicit error.

```

// programs/beezie-claw/src/instructions/initialize_factory.rs, line 26
constraint = admin.key() ==
program_data.upgrade_authority_address.unwrap_or(Pubkey::default())
    @ BeezieClawError::UnauthorizedAdmin,
// unwrap_or(Pubkey::default()) -> unsatisfiable comparison when
// upgrade_authority_address is None

```

Recommendations

It is recommended to make the failure mode an explicit error instead of a silent degenerate comparison, and operationally always initialize the factory before revoking upgrade authority